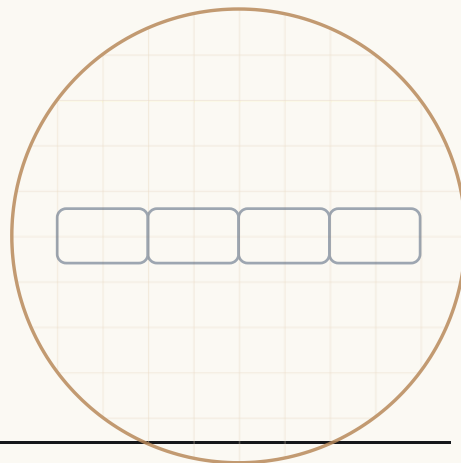


TRABAJO FINAL · UNIVERSIDAD

EduAnalytics

Secure

Implementación de una herramienta de Learning Analytics con elementos de criptografía simétrica y simulación blockchain para registros académicos.



ASIGNATURA

Análisis de Datos II
Criptografía, Blockchain y Learning Analytics

ALUMNO

Manuel Antón Herrerías

FECHA DE ENTREGA

Mayo 2026

STACK TÉCNICO

Python 3.13

Streamlit

Fernet/AES

SHA-256

Scikit-learn

00 Índice

01	Introducción y objetivos	03
1.1	Tecnologías utilizadas	03
02	Descripción del proyecto	04
2.1	Arquitectura modular de la herramienta	04
2.2	Dataset sintético	05
03	Implementación criptográfica (AES / Fernet)	06
3.1	Fundamentos de AES y Fernet	06
3.2	Implementación del módulo	06
3.3	Demostración en la herramienta	07
3.4	Justificación: AES vs RSA	08
04	Simulación blockchain con SHA-256	09
4.1	Fundamentos de la cadena de bloques	09
4.2	Implementación del módulo	09
4.3	Propiedades de SHA-256 y efecto avalancha	10
05	Learning Analytics y predicción de riesgo	11
5.1	Análisis estadístico y visualizaciones	11
5.2	Modelo predictivo de riesgo académico	12
5.3	Métricas de evaluación	13
06	Justificación de decisiones técnicas	14
07	Metodología y proceso de desarrollo	15
08	Conclusiones	16
09	Referencias	17

01 Introducción y objetivos

Las plataformas de aprendizaje online gestionan hoy en día volúmenes enormes de datos educativos: notas, registros de asistencia, participación en clase, entregas de tareas y patrones de comportamiento del alumnado. Toda esta información resulta enormemente valiosa para identificar estudiantes en riesgo y personalizar la intervención pedagógica, pero al mismo tiempo representa un activo sensible que exige mecanismos sólidos de protección y garantías de integridad.

La ausencia de estos mecanismos genera dos problemas concretos. Por un lado, la exposición de datos personales de menores o adultos en situación académica vulnerable supone un riesgo real de privacidad que contraviene el RGPD. Por otro, la manipulación de registros académicos —desde calificaciones hasta actas de evaluación— puede perpetrarse sin dejar rastro cuando los sistemas no incorporan trazabilidad criptográfica.

EduAnalytics Secure es la herramienta desarrollada para este trabajo. Su objetivo es demostrar, de forma práctica y ejecutable, cómo integrar tres tecnologías complementarias en un único sistema de learning analytics:

- **Cifrado AES** (implementado mediante `Fernet` de la librería `cryptography`) para proteger los datos personales de los estudiantes.
- **Simulación blockchain** basada en cadenas de hashes SHA-256 para garantizar la inmutabilidad e integridad de los registros académicos.
- **Análisis estadístico** y modelo de predicción de riesgo académico con regresión logística, incluyendo visualizaciones interactivas.

La herramienta está dirigida principalmente a **profesores y administradores educativos** que necesiten acceder a información de rendimiento garantizando que los datos sensibles permanecen protegidos y que ningún registro ha sido alterado sin autorización.

“Cifrar los datos de un estudiante no es solo una decisión técnica; es una decisión ética con consecuencias sobre su privacidad y sus derechos.”

1.1 Tecnologías utilizadas

COMPONENTE	LIBRERÍA / TECNOLOGÍA	VERSIÓN
Interfaz de usuario	Streamlit	1.57+
Cifrado simétrico	cryptography (Fernet / AES)	48.0+
Hash / Blockchain	hashlib (stdlib)	Python 3.13
Análisis de datos	Pandas + NumPy	3.0 / 2.4
Machine Learning	Scikit-learn	1.8+

02 Descripción *del proyecto*



FIG. 01 Pantalla principal de la aplicación Streamlit, con navegación por las cinco páginas y resumen de los tres bloques funcionales.

2.1 Arquitectura modular

La herramienta sigue una arquitectura modular organizada en cuatro capas independientes. Esta separación de responsabilidades facilita tanto el mantenimiento del código como la comprensión de cada componente de forma aislada:

```
herramienta/
├─ app.py                ← App principal Streamlit (5 páginas)
├─ requirements.txt      ← Dependencias del proyecto
├─ clave.key             ← Clave AES generada automáticamente
├─ data/
│   ├── generar_datos.py ← Generador de datos sintéticos
│   └─ estudiantes.csv   ← Dataset de 50 estudiantes
├─ crypto/
│   └─ cifrado_aes.py    ← Módulo de cifrado/descifrado AES
├─ blockchain/
│   └─ cadena.py         ← Simulación blockchain SHA-256
└─ analytics/
    ├── analisis.py      ← Estadísticas y detección de riesgo
    └─ prediccion.py     ← Modelo de regresión logística
```

2.2 Dataset sintético

Para poder demostrar todas las funcionalidades sin depender de datos reales, la herramienta genera un conjunto de datos sintéticos con **50 estudiantes**. Cada registro incluye los siguientes campos:

CAMPO	TIPO	DESCRIPCIÓN
id	String	Identificador único (EST001..EST050)
nombre	String (cifrable)	Nombre completo del estudiante
correo	String (cifrable)	Correo institucional
nota_media	Float (0-10)	Media de calificaciones del curso
asistencia	Float (0-100)	Porcentaje de asistencia
participacion	Float (0-10)	Nivel de participación en clase
tareas_entregadas	Float (0-100)	Porcentaje de tareas entregadas
nota_examen	Float (0-10)	Calificación del examen final
en_riesgo	Int (0/1)	Etiqueta de riesgo académico

Los datos se generan con distribuciones normales realistas (`numpy.random.normal`) y se guardan en formato CSV. La variable `en_riesgo` se calcula automáticamente: un estudiante se considera en riesgo si su nota media es inferior a 5.0 o su asistencia está por debajo del 50%.

POR QUÉ UN DATASET SINTÉTICO

Trabajar con datos sintéticos elimina cualquier riesgo de exposición de información real de estudiantes durante el desarrollo y la presentación del trabajo. Además, permite forzar la presencia de casos en riesgo, escenarios extremos y correlaciones controladas que serían difíciles de garantizar con un dataset real reducido.

03 Implementación *criptográfica*

3.1 Fundamentos de AES y Fernet

AES (Advanced Encryption Standard) es el algoritmo de cifrado simétrico estándar adoptado por el NIST en 2001. Trabaja sobre bloques de 128 bits y soporta claves de 128, 192 o 256 bits. Su seguridad a día de hoy es sólida contra ataques clásicos, aunque, como cualquier criptografía simétrica, su principal vulnerabilidad reside en la gestión de claves.

En este proyecto se utiliza **Fernet**, una implementación de alto nivel disponible en la librería `cryptography` de Python. Fernet aplica **AES-128-CBC** con un vector de inicialización (IV) aleatorio en cada operación de cifrado, y añade un **HMAC-SHA256** para autenticación del mensaje. Esto garantiza que cualquier modificación en el texto cifrado sea detectada antes del descifrado.

3.2 Implementación del módulo

El módulo `crypto/cifrado_aes.py` expone seis funciones: gestión de la clave (`generar_clave` , `cargar_clave`), cifrado y descifrado de strings (`cifrar` , `descifrar`) y dos helpers para aplicar el cifrado a columnas completas de un `DataFrame` (`cifrar_columnas` , `descifrar_columnas`). El núcleo del módulo es el siguiente:

```
import os
from cryptography.fernet import Fernet

KEY_FILE = os.path.join(os.path.dirname(os.path.abspath(__file__)), "..", "clave.key")

def generar_clave() → bytes:
    clave = Fernet.generate_key()
    with open(KEY_FILE, "wb") as f:
        f.write(clave)
    return clave

def cargar_clave() → bytes:
    if not os.path.exists(KEY_FILE):
        return generar_clave()
    with open(KEY_FILE, "rb") as f:
        return f.read()

def cifrar(texto: str, clave: bytes) → str:
    f = Fernet(clave)
    return f.encrypt(texto.encode("utf-8")).decode("utf-8")

def descifrar(texto_cifrado: str, clave: bytes) → str:
    f = Fernet(clave)
    return f.decrypt(texto_cifrado.encode("utf-8")).decode("utf-8")
```

Los campos `nombre` y `correo` del dataset se cifran fila a fila mediante `cifrar_columnas()`, que aplica `cifrar()` sobre cada valor del `DataFrame`. La clave se genera la primera vez que se ejecuta la app y se almacena localmente en `clave.key`; las siguientes ejecuciones la recuperan con `cargar_clave()`.

NOTA DE SEGURIDAD

En un entorno de producción real, esta clave debería gestionarse con un sistema de gestión de secretos (como AWS KMS o Azure Key Vault), **nunca almacenada en texto plano** junto al código.

3.3 Demostración en la herramienta

La página *Criptografía AES* de la aplicación permite demostrar el funcionamiento del cifrado de forma interactiva. El usuario puede:

- Ver la clave AES actual en formato base64.
- Introducir cualquier texto y cifrarlo al momento.
- Descifrar el resultado para verificar que la información original se recupera íntegramente.
- Activar el toggle en la página de datos para ver los nombres y correos cifrados.



FIG. 02 Página de demostración AES: gestión de la clave, cifrado y descifrado de texto arbitrario.



FIG. 03 Vista del dataset con el toggle activado: nombres y correos se sustituyen por su versión cifrada en base64.

3.4 Justificación: AES vs RSA

La elección de AES (cifrado simétrico) frente a RSA (cifrado asimétrico) para proteger los datos del dataset responde a razones técnicas y prácticas:

CRITERIO	AES (FERNET)	RSA-2048
Velocidad	Muy alta (hardware acelerado)	Baja (operaciones modulares costosas)
Uso óptimo	Cifrado masivo de datos	Intercambio de claves / firmas digitales
Tamaño cifrado	Proporcional al original	Mínimo 256 bytes por operación
Seguridad	AES-128 es seguro hoy	RSA-2048 es seguro hoy

EN RESUMEN

RSA tiene sentido cuando necesitamos que dos partes que no se conocen intercambien información de forma segura (por ejemplo, en TLS). Para cifrar los 50 registros de un dataset con un único sistema de acceso controlado, **AES es la opción correcta**: es más eficiente, produce texto cifrado más compacto y su seguridad es igualmente sólida.

04 Simulación *blockchain*

4.1 Fundamentos de la cadena de bloques

Una blockchain es, en esencia, una **lista enlazada** donde cada nodo (bloque) contiene el hash criptográfico del nodo anterior. Esta característica hace que cualquier modificación en un bloque **invalida automáticamente todos los bloques posteriores**, ya que sus hashes dejan de coincidir.

En el contexto educativo, esto permite crear un registro auditable e inmutable de las actas académicas: una vez que una calificación se incorpora a la cadena, cualquier intento de modificarla sin regenerar toda la cadena posterior se detecta al verificar la integridad.

4.2 Implementación del módulo

```
import hashlib, json, time
from dataclasses import dataclass, field

@dataclass
class Bloque:
    indice: int
    timestamp: float
    datos: dict
    hash_anterior: str
    hash: str = field(default="", init=False)

    def __post_init__(self):
        self.hash = self.calcular_hash()

    def calcular_hash(self) → str:
        contenido = json.dumps({
            "indice": self.indice,
            "timestamp": self.timestamp,
            "datos": self.datos,
            "hash_anterior": self.hash_anterior
        }, sort_keys=True, ensure_ascii=False)
        return hashlib.sha256(contenido.encode("utf-8")).hexdigest()

    def es_valido(self) → bool:
        return self.hash == self.calcular_hash()

class CadenaEducativa:
    def verificar_integridad(self) → tuple[bool, list[str]]:
        errores = []
        for i in range(1, len(self.cadena)):
            actual = self.cadena[i]
            anterior = self.cadena[i - 1]
            if not actual.es_valido():
                errores.append(f"Bloque {actual.indice}: hash interno corrupto")
            if actual.hash_anterior != anterior.hash:
                errores.append(f"Bloque {actual.indice}: enlace roto con bloque {anterior.indice}")
        return len(errores) == 0, errores
```

El uso de `@dataclass` reduce el *boilerplate* y aplica la firma de tipos a los campos del bloque. La función `__post_init__` calcula automáticamente el hash al crear cada bloque, y `es_valido()` compara el hash almacenado con el recalculado en el momento — base de la verificación de integridad.

4.3 Propiedades de SHA-256 y efecto avalancha

SHA-256 produce un hash de **256 bits (64 caracteres hexadecimales)** con las siguientes propiedades que lo hacen adecuado para blockchain:

- **Determinista:** el mismo input siempre produce el mismo hash.
- **Efecto avalancha:** un cambio mínimo en el input produce un hash completamente diferente.
- **Resistente a preimagen:** no es posible obtener el input a partir del hash.
- **Resistente a colisiones:** prácticamente imposible que dos inputs distintos produzcan el mismo hash.



FIG. 04 Verificación de integridad: la cadena reporta correctamente «Cadena íntegra» antes de cualquier manipulación.

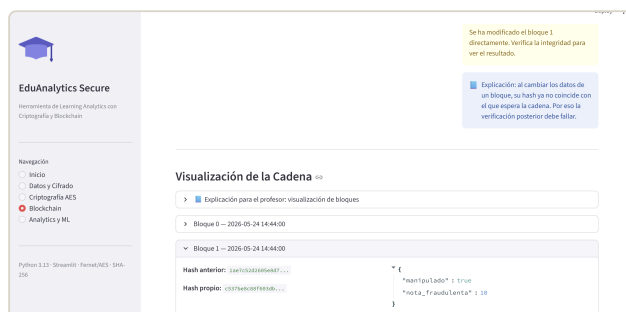


FIG. 05 Al modificar manualmente el bloque 1, el hash interno deja de coincidir con el almacenado y la integridad se rompe automáticamente.

05 Learning Analytics y predicción

5.1 Análisis estadístico y visualizaciones

El módulo `analytics/analisis.py` implementa el análisis descriptivo del dataset. Calcula media, mediana, desviación típica, mínimo y máximo para cada variable numérica, y aplica el criterio de detección de riesgo sobre el conjunto completo.

La herramienta incluye cuatro visualizaciones que permiten al docente interpretar los resultados de un vistazo:

- **Histograma de notas medias** diferencia los grupos *sin riesgo* y *en riesgo*, con línea de umbral en 5.0.
- **Dispersión asistencia vs. nota media** permite identificar la correlación y localizar a los estudiantes en riesgo (puntos rojos).
- **Mapa de calor de correlaciones** (Seaborn) muestra qué factores tienen más influencia en el rendimiento.
- **Barras de participación por grupo** compara la participación media de ambos grupos de riesgo.

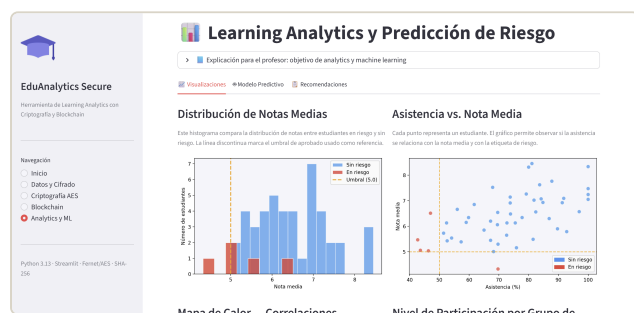


FIG. 06 Distribución de notas medias y dispersión asistencia vs. nota. Los estudiantes en riesgo (rojo) se concentran en la zona inferior izquierda.

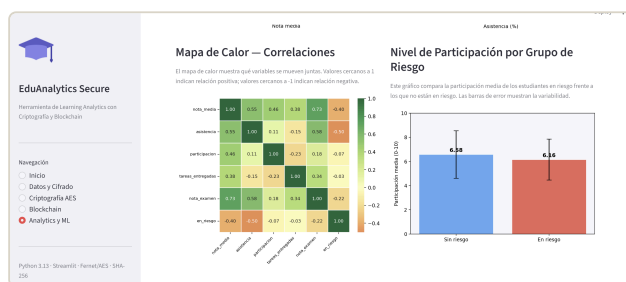


FIG. 07 Mapa de calor de correlaciones y participación media por grupo. La asistencia tiene la correlación negativa más fuerte con la etiqueta de riesgo (-0.50).

5.2 Modelo predictivo de riesgo académico

Para la predicción de riesgo se implementa una **regresión logística** (`LogisticRegression` de Scikit-learn). La elección de este algoritmo frente a alternativas como árboles de decisión o SVM responde a su **interpretabilidad**: permite obtener los coeficientes de cada variable y explicar al docente por qué el sistema considera que un estudiante está en riesgo.

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

FEATURES = ["asistencia", "participacion", "tareas_entregadas", "nota_examen"]
TARGET = "en_riesgo"

def entrenar_modelo(df: pd.DataFrame):
    X = df[FEATURES].values
    y = df[TARGET].values
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.3, random_state=42,
        stratify=y if y.sum() >= 2 else None
    )
    scaler = StandardScaler() # Normalizar para igualar escalas
    X_train_s = scaler.fit_transform(X_train)
    X_test_s = scaler.transform(X_test)
    modelo = LogisticRegression(random_state=42, max_iter=1000)
    modelo.fit(X_train_s, y_train)
    return modelo, scaler, X_test_s, y_test
```

Los datos se escalan con `StandardScaler` para evitar que variables con rangos muy distintos (por ejemplo, asistencia en porcentaje vs. participación en escala 0-10) dominen el modelo. El dataset se divide en **70% entrenamiento** y **30% prueba**, manteniendo la proporción de la clase minoritaria mediante `stratify`. La función devuelve tanto el modelo entrenado como el *scaler* ajustado y el conjunto de test ya escalado, listos para el cálculo de métricas.

5.3 Métricas de evaluación

Los resultados del modelo sobre el conjunto de prueba (15 muestras) son los siguientes:

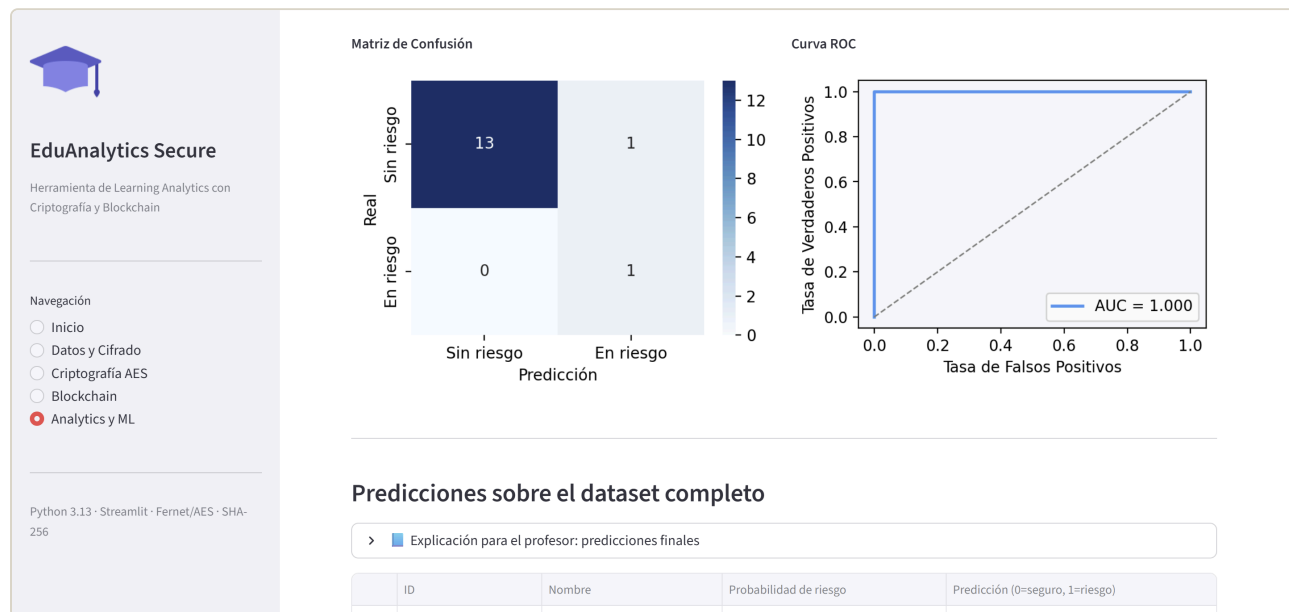


FIG. 08 Matriz de confusión (izq.) y curva ROC (der.) del modelo sobre el conjunto de test. AUC = 1.000.

POR QUÉ PRIORIZAR RECALL

En este contexto educativo, preferimos generar algunas alertas falsas antes que dejar sin detectar a un estudiante que realmente necesita ayuda. Un *Recall* de 1.0 garantiza que ningún estudiante en riesgo queda invisible para el sistema. La *accuracy* puede ser engañosa con clases desequilibradas: un modelo que siempre predijera «sin riesgo» en una población con 10% de riesgo real tendría 90% de accuracy y un recall de 0.

06 Justificación de *decisiones técnicas*

6.1 Elección de Streamlit como interfaz

Streamlit permite construir aplicaciones web interactivas directamente desde Python, sin necesidad de conocimientos de HTML, CSS o JavaScript. Para un prototipo académico como este, donde la prioridad es demostrar las capacidades de análisis y criptografía, Streamlit ofrece la mejor relación entre coste de desarrollo y calidad de presentación. La alternativa (Flask / Django) habría triplicado el tiempo de desarrollo sin añadir valor al contenido académico.

6.2 SHA-256 como base de la simulación blockchain

Se utiliza `hashlib` de la biblioteca estándar de Python, sin dependencias externas, porque el objetivo es demostrar el **principio de encadenamiento mediante hashes**, no emular un blockchain completo con consenso distribuido. Un blockchain real (Ethereum, Hyperledger) requeriría una red de nodos, mecanismos de consenso y contratos inteligentes, complejidades que exceden el alcance de este trabajo pero que la herramienta referencia conceptualmente.

6.3 Consideraciones de seguridad y privacidad

- La clave AES se genera con `Fernet.generate_key()`, que usa `os.urandom()` internamente. Nunca se expone en el código fuente.
- Los datos sintéticos no contienen información real de ninguna persona. En un entorno de producción, los datos reales deberían anonimizarse según el principio de *minimización de datos* del RGPD.
- El modelo predictivo genera **recomendaciones de intervención, no decisiones automáticas**. La supervisión humana es explícitamente requerida en la interfaz.
- No se almacenan contraseñas en claro; si se implementara autenticación, se usarían funciones KDF como `Argon2` o `bcrypt` (nunca SHA-256 directo sobre la contraseña).

07 Metodología y proceso

El desarrollo de EduAnalytics Secure sigue un proceso iterativo dividido en cuatro fases:

FASE 1 · DISEÑO Y PLANIFICACIÓN

Se partió de los requisitos del enunciado y se definió la **arquitectura modular**: separar la lógica de cifrado, blockchain y analytics en módulos independientes facilita las pruebas unitarias y permite que cada componente evolucione de forma autónoma.

FASE 2 · GENERACIÓN DE DATOS Y MÓDULOS BASE

Se implementó primero el generador de datos sintéticos para disponer de un dataset realista con el que probar el resto de módulos. Las distribuciones normales (`np.random.normal`) con los parámetros ajustados manualmente producen datos con correlación realista entre variables como asistencia y nota media.

FASE 3 · IMPLEMENTACIÓN DE CRIPTOGRAFÍA Y BLOCKCHAIN

Los módulos `cifrado_aes.py` y `cadena.py` se desarrollaron con pruebas unitarias integradas: se cifró y descifró un texto de prueba para verificar la integridad del cifrado AES, y se construyó una cadena de 3 bloques para verificar que la modificación de un bloque central rompía la cadena completa.

FASE 4 · INTEGRACIÓN EN STREAMLIT Y AJUSTE DE VISUALIZACIONES

La integración en la app de Streamlit permitió detectar algunos problemas menores: el estado de la cadena blockchain y el modelo entrenado debían persistir en `st.session_state` para no perderse al navegar entre páginas. Esto ilustra un comportamiento específico de Streamlit (*rerun* completo del script en cada interacción) que difiere del flujo estándar de aplicaciones web con estado en servidor.

08 Conclusiones

El desarrollo de EduAnalytics Secure ha permitido conectar de forma práctica los tres bloques temáticos de la asignatura: criptografía, blockchain y learning analytics. Lo que en los apuntes aparecía como conceptos separados cobra sentido cuando se integran en un sistema funcional con un problema real.

Quizá el aprendizaje más importante del proyecto sea entender que la criptografía no es solo un mecanismo técnico, sino una **herramienta ética**. Cifrar los datos de un estudiante es una decisión que tiene consecuencias sobre su privacidad y sus derechos. Del mismo modo, la simulación blockchain deja claro que la inmutabilidad de un registro no lo hace correcto: si el dato original es erróneo, el blockchain solo lo perpetúa con más firmeza.

En cuanto al modelo predictivo, el trabajo refuerza la idea expresada en los foros: un modelo con *Recall* 1.0 y AUC-ROC perfecto sobre datos sintéticos equilibrados no garantiza el mismo rendimiento sobre datos reales, donde el desequilibrio entre clases, el ruido y los factores contextuales no capturados por las variables disponibles degradan necesariamente el rendimiento. **La supervisión humana no es un complemento del sistema: es una condición ética irrenunciable.**

8.1 Limitaciones y mejoras futuras

- Integración con un blockchain real (Ethereum o Hyperledger Fabric) para eliminar la centralización de la cadena simulada.
- Gestión de claves mediante un HSM (Hardware Security Module) o servicio en la nube (AWS KMS).
- Sustitución de los datos sintéticos por datos reales anonimizados de una plataforma LMS (Moodle, Canvas).
- Extensión del modelo con algoritmos más sofisticados (XGBoost, LSTM para datos secuenciales de acceso) y validación cruzada.
- Implementación de autenticación de usuarios con diferenciación de roles (profesor vs. administrador).

09 Referencias

1. Python Cryptography Library. cryptography.io/en/latest/
2. Python `hashlib` — Módulo de hashing seguro. docs.python.org/3/library/hashlib.html
3. Pandas Documentation. pandas.pydata.org/docs/
4. Scikit-learn: Machine Learning in Python. scikit-learn.org/stable/
5. Streamlit Documentation. docs.streamlit.io
6. JISC (2015). *Code of Practice for Learning Analytics*. jisc.ac.uk/guides/code-of-practice-for-learning-analytics
7. NIST (2001). *Advanced Encryption Standard (AES)*. FIPS PUB 197.
8. Nakamoto, S. (2008). *Bitcoin: A Peer-to-Peer Electronic Cash System*.
9. Siemens, G. (2012). *Learning Analytics: Envisioning a Research Discipline and a Domain of Practice*.
10. Reglamento (UE) 2016/679 — *Reglamento General de Protección de Datos (RGPD)*.

FIN DEL DOCUMENTO

EduAnalytics Secure

Manuel Antón Herrerías · Mayo 2026

Análisis de Datos II
Criptografía · Blockchain
Learning Analytics